

Cours 23 — Fichiers, périphériques et droits

Semaine 23 ■ Manuel Hachette ch. 12 (§ 12.2, 12.3) ■ 2 × 1 h

Objectifs

- Décrire le système de gestion de fichiers : arborescence, chemins
- Lire et modifier les droits d'accès (r, w, x)
- Situer les périphériques de stockage et leurs ordres de grandeur

1. Des octets aux fichiers

Un disque ne connaît que des octets numérotés. Le **système de gestion de fichiers** (une mission de l'OS) les organise en **fichiers** nommés, rangés dans des **répertoires** qui forment un **arbre** unique, enraciné à / :

```
/
|-- home/
|   |-- eleve/
|       |-- nsi/
|           |-- tp.py
|-- bin/
|-- etc/
```

- **Chemin absolu** : depuis la racine — /home/eleve/nsi/tp.py ; commence toujours par /.
- **Chemin relatif** : depuis le répertoire courant — si je suis dans /home/eleve : nsi/tp.py ; .. désigne le parent, . le répertoire courant.

Un fichier n'est qu'une **suite d'octets** : c'est son format (et le programme qui le lit) qui en fait un texte UTF-8 (semaine 6), une image, un exécutable.

2. Les droits d'accès

Système multi-utilisateurs → chaque fichier a un **propriétaire**, un **groupe**, et des droits pour trois publics : propriétaire (u), groupe (g), autres (o). Trois droits chacun : **read** (lire), **write** (écrire), **execute** (exécuter ; pour un répertoire : le traverser).

C'est la première colonne de `ls -l` :

```
-rw-r--r--  1 eleve nsi   1284 mar  3 10:12 tp.py
drwxr-x---  2 eleve nsi   4096 mar  3 10:15 projet
```

Lecture : - ou d (fichier/répertoire), puis trois triplets **rw**x. Ici **tp.py** : le propriétaire lit et écrit, groupe et autres lisent seulement.

Modifier : `chmod` —

```
chmod u+x script.sh      le propriétaire peut exécuter
chmod go-r secret.txt    groupe et autres ne lisent plus
```

(La notation octale `chmod 640` existe : r=4, w=2, x=1 — du binaire à trois bits, semaine 5!)

3. Les périphériques de stockage

support	accès typique	particularité
disque dur (HDD)	≈ 10 ms	mécanique, grande capacité
SSD	$\approx 0,1$ ms	mémoire flash, silencieux
RAM	≈ 100 ns	volatile (s'efface sans courant)

Hierarchie : registres > caches > RAM > SSD/disque — plus c'est rapide, plus c'est petit et cher. D'où l'importance d'**enregistrer** (RAM volatile!) et le rôle des caches.

À retenir

- Arborescence unique depuis / ; chemin absolu (depuis /) vs relatif (depuis ici ; .. = parent).
- `ls -l` : type + trois triplets rwx pour u, g, o ; modification par `chmod`.
- x sur un répertoire = le traverser.
- RAM volatile et rapide ; SSD/disque persistants et lents — hiérarchie de mémoires.